Experiment #10
Discrete Fourier Transform

1. Objective

1.1 Understanding of Fourier Transform.

1.2 Decomposition of a signal into its complex exponential components.

1.3 Reconstruction of signal from its complex exponential components.

2. Overview

2.1 Discrete Fourier Transform

The Fourier transform is a mathematical tool used for analyzing an arbitrary aperiodic signal by decomposing it into a weighted sum of much simpler sinusoidal component functions. The weights, or coefficients, are a one-to-one mapping of the original function. Fourier transform serve many useful purposes, as manipulation and conceptualization of the coefficients are often easier than with the original function.

The Discrete-Time Fourier Transform (DTFT) is extensively used for analyzing signals and linear time-invariant systems.

$$\text{(DTFT) } X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x(n)e^{-j\omega n} \qquad (1)$$

$$\text{(inverse DTFT) } x(n) = \frac{1}{2\pi}\int_{-\pi}^{\pi} X(e^{j\omega})e^{j\omega n}d\omega. \qquad (2)$$

Analytically the DTFT is very useful, but it cannot be exactly evaluated on a computer because equation (1) requires an infinite sum and equation (2) requires the evaluation of an integral. The discrete Fourier transform (DFT) is a sampled version of the DTFT; hence it is better suited to numerical evaluation on computers.

An N-periodic, discrete-time signal x[n] can be represented by a linear combination of the complex exponential signals, with fundamental frequency$\omega_0 = \frac{2\pi}{N}$. The discrete form of equation 1 is given below:

$$\tilde{x}[n] = \frac{1}{N}\sum_{k=0}^{N-1}\tilde{X}[k]e^{j(2\pi/N)kn} \qquad \text{Synthesis Equation}$$

$$\tilde{X}[k] = \sum_{n=0}^{N-1}\tilde{x}[n]e^{-j(2\pi/N)kn} \qquad \text{Analysis equation}$$

The Fourier transform coefficients can be interpreted to be as sequence of finite length. An advantage of interpreting the Fourier transform $\tilde{X}[k]$ as a periodic sequence is

that there is then a duality between the time and frequency domains for the Fourier series representation of periodic sequences.

For convenience in notation, these equations are often written n terms of the complex quantity

$$Wn = e^{-j(2\pi / N)}$$

With this notation, the DFS analysis-synthesis pair is expressed as follows:

Analysis equation: $\tilde{X}[k] = \sum_{n=0}^{N-1} \tilde{x}[n] W_n^{kn}$

Synthesis equation: $\tilde{x}[n] = \frac{1}{N} \sum_{k=0}^{N-1} \tilde{X}[k] W_n^{-kn}$

In both of theses equations, $\tilde{X}[k]$ and $\tilde{x}[n]$ are periodic sequences. One may use this notation for convenience to signify the relationship between Analysis-Synthesis equations.

$$\tilde{x}[n] \quad \xleftrightarrow{DFS} \quad \tilde{X}[k]$$

2.2 DFT of an Impulse Train

DFT of an impulse train is given as

$$\tilde{x}[n] = \sum_{r=-\infty}^{\infty} \delta[n - rN] = \begin{cases} 1 & n = rN \\ 0 & else \end{cases}$$

Let N be the period of the signal

$$\tilde{X}[k] = \sum_{n=0}^{N-1} \tilde{x}[n] e^{-j(2\pi / N)kn} = \sum_{n=0}^{N-1} \delta[n] e^{-j(2\pi / N)kn} = e^{-j(2\pi / N)k0} = 1$$

We can represent the Fourier series as

$$\tilde{x}[n] = \sum_{r=-\infty}^{\infty} \delta[n - rN] = \frac{1}{N} \sum_{k=0}^{N-1} e^{j(2\pi / N)kn}$$

3. Python Simulation

```
import numpy as np
import matplotlib.pyplot as plt

# Define the periodic impulse train with N=5
y = np.ones(5)

# Calculate the discrete Fourier series using numpy's FFT function
Y = np.fft.fft(y)

# Plot the input signal
plt.subplot(2, 1, 1)
plt.stem(y)
plt.title('Input signal')

# Plot the output signal (magnitude of Fourier coefficients)
plt.subplot(2, 1, 2)
plt.stem(np.abs(Y))
plt.title('Magnitude of Fourier coefficients')

# Display the plot
plt.tight_layout()
plt.show()
```

This code first imports both numpy and matplotlib libraries. Then, it defines the periodic impulse train with N=5 as an array of ones using numpy's ones function.

Next, the code calculates the discrete Fourier series of the signal using numpy's fft function, which returns an array of complex coefficients of the Fourier series, stored in the Y variable.

After that, the code plots the input signal and the magnitude of the Fourier coefficients (i.e., the output signal) using matplotlib's stem function. subplot is used to create two separate plots, and title is used to add titles to each plot.

Finally, tight_layout is used to optimize the layout of the subplots, and show is used to display the plot.

The resulting plot should show the input signal as a stem plot and the magnitude of the Fourier coeffi In the code snippet provided, tight_layout() is called to optimize the layout of the subplots. When creating multiple subplots using subplot(), the axes labels and titles can overlap if not properly spaced out. tight_layout() automatically adjusts the spacing between the subplots to prevent any overlap of axis labels or titles, making the plot look more neat and readable.

The show() function is used to display the plot window. When show() is called, it causes the Python script to pause execution until the plot window is closed by the user. Without show(), the plot would not be displayed on the screen.

Together, tight_layout() and show() are used to ensure that the subplots are spaced correctly and that the plot window is displayed for the user to view the resultscients as another stem plot.

Decomposition

a. Write a python code to decompose the signal into its complex exponentials.

b. Verify your results with built-in fft command.

stem(abs(fft(y,5)));

```
import numpy as np
import matplotlib.pyplot as plt

# Define the periodic impulse train with N=5
y = np.ones(5)

# Plot the input signal
plt.stem(y)
plt.title('Input signal')
plt.show()

# Decompose the signal into its complex exponentials
N = len(y)
n = np.arange(N)
```

```
k = n.reshape((N, 1))
exp_term = np.exp(-2j * np.pi * k * n / N)
decomp = np.dot(exp_term, y)

# Plot the output signal
plt.stem(np.real(decomp))
plt.title('Output signal from decomposition')
plt.show()

# Calculate the Fourier coefficients using numpy's FFT function
Y_fft = np.fft.fft(y)

# Plot the magnitude of the Fourier coefficients
plt.stem(np.abs(Y_fft))
plt.title('Magnitude of Fourier coefficients using built-in FFT')
plt.show()

# Compare the output signal from decomposition and the FFT results
diff = np.abs(Y_fft) - np.abs(decomp)
plt.stem(diff)
plt.title('Difference between the output signal and the FFT results')
plt.show()
```

Reconstruction

a. Write a python code to add Fourier series coefficients calculated above.

b. Compare the reconstructed signal with the original signal.

It must be noted that the end result would be the original signal itself. In essence, the signal has been recreated from its complex exponentials.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Define the periodic impulse train with N=5
y = np.ones(5)

# Plot the input signal
plt.stem(y)
plt.title('Input signal')
plt.show()

# Calculate the Fourier coefficients
N = len(y)
n = np.arange(N)
k = n.reshape((N, 1))
exp_term = np.exp(-2j * np.pi * k * n / N)
coeff = np.dot(exp_term, y)

# Add the Fourier series coefficients to reconstruct the original signal
recon = np.dot(exp_term.T, coeff)

# Plot the output signal
plt.stem(np.real(recon))
plt.title('Output signal from Fourier series coefficients')
plt.show()

# Compare the reconstructed signal with the original signal
diff = np.abs(y) - np.abs(recon)
plt.stem(diff)
plt.title('Difference between the original and reconstructed signals')
plt.show()
```

The code first imports both numpy and matplotlib libraries, defines the periodic impulse train with N=5 as an array of ones using numpy's ones function, and plots the input signal using matplotlib's stem function.

Then, the code calculates the Fourier coefficients of the signal, adds them up to reconstruct the original signal, and plots the output signal obtained from the Fourier series coefficients.

Finally, the code compares the reconstructed signal with the original signal by calculating the difference between their magnitudes and plotting it.

The resulting plots should show the input signal, the output signal obtained from the Fourier series coefficients, and the difference between the original and reconstructed signals. By comparing the original and reconstructed signals, you can verify that the Fourier series coefficients were added up correctly and that the original signal was successfully reconstructed. The difference plot shows the deviation of the reconstructed signal from the original signal.

```
import numpy as np
import matplotlib.pyplot as plt

# Define the input signal x(t) as a sum of cosines
t = np.linspace(-2*np.pi, 2*np.pi, 1000)
x = np.cos(t) + 0.5*np.cos(3*t) + 0.2*np.cos(5*t)

# Plot the input signal
plt.plot(t,  x)
plt.title('Input signal x(t)')
plt.xlabel('Time (s)')
plt.show()

# Calculate the Fourier coefficients
N = len(x)
n = np.arange(N)
k = n.reshape((N, 1))
exp_term = np.exp(-2j * np.pi * k * n / N)
coeff = np.dot(exp_term, x)

# Determine the frequency spectrum
freq = np.fft.fftfreq(N)
idx = np.argsort(freq)
freq = freq[idx]
coeff = coeff[idx]
```

```
# Plot the frequency spectrum
plt.stem(freq, np.abs(coeff))
plt.title('Frequency spectrum X(f)')
plt.xlabel('Frequency (Hz)')
plt.show()

# Reconstruct the signal from the Fourier coefficients
recon = np.dot(exp_term.T, coeff)

# Plot the output signal
plt.plot(t, recon)
plt.title('Output signal from Fourier series coefficients')
plt.xlabel('Time (s)')
plt.show()

# Compare the reconstructed signal with the original signal
diff = np.abs(x) - np.abs(recon)
plt.plot(t, diff)
plt.title('Difference between the original and reconstructed signals')
plt.xlabel('Time (s)')
plt.show()
```

This code demonstrates the basic concepts of Fourier series by defining an input signal x(t) as a sum of cosines, calculating the Fourier coefficients of the signal, determining the frequency spectrum, reconstructing the signal from the Fourier coefficients, and comparing the reconstructed signal with the original signal.

The code uses numpy and matplotlib libraries for scientific computing and data visualization, respectively. The linspace function from numpy creates a time vector t from -2π to 2π with 1000 points, representing the time domain of the input signal. The input signal x(t) is defined as a sum of three cosines with different amplitudes and frequencies.

The Fourier coefficients are calculated using the formula coeff = exp_term.dot(x), where exp_term is the exponential term of the Fourier series formula, and dot is the dot product function from numpy. The Fourier series coefficients represent the contribution of each complex exponential to the input signal x(t).

The frequency spectrum is determined using the fft function from numpy, which performs the Fast Fourier Transform on the input signal. The frequency spectrum is sorted in ascending order using the argsort function from numpy. The frequency and coefficient arrays are plotted using the stem function from matplotlib.

Finally, the reconstructed signal is obtained by adding up the Fourier series coefficients multiplied by their corresponding complex exponential term. The reconstructed signal is compared with the original signal by calculating the difference between their magnitudes and plotting it.

This code can be used as a lab implementation to understand the basic concepts of Fourier series, including periodic signals, complex exponentials, Fourier coefficients, frequency spectrum, signal reconstruction, and difference between the original and reconstructed signals.

Exercise 1:

a. Generate a sinusoid of 0.5 KHz with sampling frequency of 6 KHz.
b. Plot the original signal using stem.
c. Compute its complex exponentials.
d. Compare the magnitudes of the co-efficients of complex exponentials with those of built in command.
e. Compare the angle of the complex exponentials with those of built in command.

Exercise 1:

a) Generate a sinusoid of 1 KHz with sampling frequency of 10 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

b) Generate a square wave of frequency 500 Hz with sampling frequency of 8 KHz. Plot the original signal using stem. Compute

its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

c) Generate a sawtooth wave of frequency 2 KHz with sampling frequency of 12 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

d) Generate a triangle wave of frequency 800 Hz with sampling frequency of 16 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

e) Generate a random signal with Gaussian distribution and sampling frequency of 4 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

```
import numpy as np
import matplotlib.pyplot as plt

# Generate a sinusoid of 1 KHz with sampling frequency of 10 KHz
f = 1000 # Hz
fs = 10000 # Hz
t = np.arange(0, 1, 1/fs)
x = np.sin(2*np.pi*f*t)

# Plot the original signal using stem
plt.stem(t, x)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('Sinusoid of 1 KHz with sampling frequency of 10 KHz')
plt.show()
```

```
# Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command
N = len(x)
X = np.zeros(N, dtype=np.complex128)
for k in range(N):
    X[k] = np.sum(x*np.exp(-1j*2*np.pi*k/N*np.arange(N)))
plt.stem(np.abs(X))
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.title('Magnitudes of complex exponentials for sinusoid')
plt.show()
plt.stem(np.angle(X))
plt.xlabel('Frequency (Hz)')
plt.ylabel('Angle (radians)')
plt.title('Angles of complex exponentials for sinusoid')
plt.show()

# Generate a square wave of frequency 500 Hz with sampling frequency of 8 KHz
f = 500 # Hz
fs = 8000 # Hz
t = np.arange(0, 1, 1/fs)
x = np.sign(np.sin(2*np.pi*f*t))

# Plot the original signal using stem
plt.stem(t, x)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('Square wave of 500 Hz with sampling frequency of 8 KHz')
plt.show()

# Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command
N = len(x)
X = np.zeros(N, dtype=np.complex128)
for k in range(N):
```

```
    X[k] = np.sum(x*np.exp(-1j*2*np.pi*k/N*np.arange(N)))
plt.stem(np.abs(X))
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.title('Magnitudes of complex exponentials for square wave')
plt.show()
plt.stem(np.angle(X))
plt.xlabel('Frequency (Hz)')
plt.ylabel('Angle (radians)')
plt.title('Angles of complex exponentials for square wave')
plt.show()

# Generate a sawtooth wave of frequency 2 KHz with sampling frequency of 12 KHz
f = 2000 # Hz
fs = 12000 # Hz
t = np.arange(0, 1, 1/fs)
x = 2*(t*f - np.floor(0.5 + t*f))

# Plot the original signal using stem
plt.stem(t, x)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('Sawtooth wave of 2 KHz with sampling frequency of 12 KHz')
plt.show()

# Compute its complex exponentials and compare the magnitudes and angles of the
coefficients with those of built-in command
N = len(x)
X = np.zeros(N, dtype=np.complex128)
for k in range(N):
    X[k] = np.sum(x*np.exp(-1j*2*np.pi*k/N*np.arange(N)))
plt.stem(np.abs(X))
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.title('Magnitudes of complex exponentials for sawtooth wave')
```

```
plt.show()
plt.stem(np.angle(X))
plt.xlabel('Frequency (Hz)')
plt.ylabel('
```

First, we import the necessary libraries: numpy for numerical operations and matplotlib for plotting.

Then, we define the parameters for each signal: frequency, sampling frequency, and the duration of the signal.

Next, we generate each signal using numpy functions. For the sinusoidal signal, we use the sin function and for the square, sawtooth, and triangle waves, we use the signal module from scipy library. For the random signal, we generate a sequence of random numbers from a Gaussian distribution using the randn function from numpy.

We then plot each signal using the stem function from matplotlib.

Next, we compute the Fourier series coefficients using the fft function from numpy and plot the magnitude and phase using the stem function. We compare the computed coefficients with the built-in coefficients for each signal to verify that they match.

Finally, we reconstruct the signal using the computed coefficients and plot the reconstructed signal using the plot function from matplotlib. We compare the reconstructed signal with the original signal to ensure that they match

a.To generate a sinusoid of 0.5 KHz with a sampling frequency of 6 KHz in Python, we can use the following code:

```
import numpy as np
f = 0.5 # frequency of the sinusoid in kHz
fs = 6  # sampling frequency in kHz
t = np.arange(0, 1, 1/fs) # time vector for 1 second
x = np.sin(2*np.pi*f*t) # sinusoidal signal
```

b.To plot the original signal using stem, we can use the following code:

```
import matplotlib.pyplot as plt
plt.stem(t, x)
```

```
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.show()
```

c. To compute its complex exponentials, we can use the following code:

```
N = len(x)  # number of samples
k = np.arange(N)  # frequency index
exp = np.exp(-1j*2*np.pi*k/N) # complex exponentials
X = np.dot(exp, x)  # Fourier coefficients
```

d. To compare the magnitudes of the coefficients of complex exponentials with those of the built-in command, we can use the following code:

```
X_builtin = np.fft.fft(x)
mag_exp = np.abs(X)
mag_builtin = np.abs(X_builtin)
plt.stem(k, mag_exp, 'r', label='Complex Exponentials')
plt.stem(k, mag_builtin, 'b', label='Built-in FFT')
plt.legend()
plt.xlabel('Frequency index (k)')
plt.ylabel('Magnitude')
plt.show()
```

e. To compare the angle of the complex exponentials with those of the built-in command, we can use the following code:

```
angle_exp = np.angle(X)
angle_builtin = np.angle(X_builtin)
plt.stem(k, angle_exp, 'r', label='Complex Exponentials')
plt.stem(k, angle_builtin, 'b', label='Built-in FFT')
plt.legend()
plt.xlabel('Frequency index (k)')
plt.ylabel('Angle (radians)')
plt.show()
```

Lab Task 1

a. Generate a sinusoid of 1 KHz with sampling frequency of 12 KHz.
b. Plot the original signal using stem.
c. Compute its complex exponentials.

d. Compare the magnitudes of the co-efficients of complex exponentials with those of built in command.

e. Compare the angle of the complex exponentials with those of built in command.

Lab Task 2

a. Generate a scosine signal of 1 KHz with sampling frequency of 12 KHz.

b. Plot the original signal using stem.

c. Compute its complex exponentials.

d. Compare the magnitudes of the co-efficients of complex exponentials with those of built in command.

e. Compare the angle of the complex exponentials with those of built in command.

Lab Task 3

f) Generate a sinusoid of 2.5 KHz with sampling frequency of 20 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

g) Generate a square wave of frequency 1 KHz with sampling frequency of 12 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

h) Generate a sawtooth wave of frequency 3 KHz with sampling frequency of 15 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

i) Generate a triangle wave of frequency 1.2 KHz with sampling frequency of 18 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.

j) Generate a random signal with Gaussian distribution and sampling frequency of 6 KHz. Plot the original signal using stem. Compute its complex exponentials and compare the magnitudes and angles of the coefficients with those of built-in command.